

🔗 See Topic 13 → Backtracking Masterclass

🎯 **GOAL:** Master Dynamic Programming! Learn the 8-step `SC-RBR-MTS` Framework, Top-Down Memoization vs Bottom-Up Tabulation, Space Optimization tricks, 1D/2D/Grid DP, Knapsack, LCS, LIS, and solve all NeetCode DP problems!

📦 REUSABLE DYNAMIC PROGRAMMING UTILITIES

Memoization Table Initialization:

```
// 1D Memo initialization with -1
int[] memo = new int[n + 1];
Arrays.fill(memo, -1);

// 2D Memo initialization with -1
int[][] memo2D = new int[n + 1][m + 1];
for (int[] row : memo2D) Arrays.fill(row, -1);
```

Space Optimization Row Swap:

```
// 1D DP array space optimization (prev vs curr)
int[] prev = new int[m + 1];
int[] curr = new int[m + 1];
// At end of outer loop step:
prev = curr.clone();
```

💡 AHA MOMENT #1: THE 2 REQUISITES FOR DYNAMIC PROGRAMMING

Dynamic Programming is simply ****Recursion + Caching (Memoization)**** OR ****Iterative Table Filling (Tabulation)****! It applies **ONLY** when a problem exhibits:

- Overlapping Subproblems:** The same smaller subproblems are computed repeatedly (e.g. `fib(3)` called multiple times in `fib(5)`).
- Optimal Substructure:** The optimal solution to the problem can be constructed from optimal solutions of its subproblems!

1 CORE DP TERMINOLOGY

- State Definition (`dp[i]`):** Mathematical description of what index `i` represents.
- Recurrence Relation:** Equation linking current state `dp[i]` to smaller subproblems.
- Base Cases:** Trivial starting conditions (e.g., `dp[0] = 0, dp[1] = 1`).
- Space Optimization:** Keeping only `prev1` & `prev2` variables instead of full $O(N)$ array!

2 MEMOIZATION VS TABULATION MATRIX

Feature	Top-Down (Memoization)	Bottom-Up (Tabulation)
Approach	Recursive (Root → Base)	Iterative (Base → Goal)
Data Structure	HashMap / `Integer[] memo`	1D / 2D Primitive Array
Call Overhead	Stack Frame Overhead	✅ Zero Stack Overhead (Fast!)
Space Optimization	Hard to optimize space	✅ Easy to compress to $O(1)$ space

🧠 VISUALIZING EXPONENTIAL RECURSION $O(2^N)$ VS DP TABULATION $O(N)$ (FIBONACCI)

EXPONENTIAL RECURSION TREE ($O(2^N)$ - RECOMPUTING FIB(2) 3 TIMES!): fib(5) / \ fib(4) fib(3) / \ / \ fib(3) fib(2) fib(2) fib(1) **DP TABULATION ARRAY ($O(N)$ TIME, $O(1)$ SPACE):** State: $dp[i] = dp[i-1] + dp[i-2]$ Array: [0, 1, 1, 2, 3, 5] → Instant lookup in linear time!

🔑 **KEY TAKEAWAY:** Always start by writing the Top-Down recursion equation, then convert it to a Bottom-Up iterative table!

💡 **AHA MOMENT #2: THE SYSTEMATIC SC-RBR-MTS FRAMEWORK**

Never code DP blindly! Follow the 8-step `SC-RBR-MTS` progression:

State → **Choice** → **Recurrence** → **Base Case** → **Recursion** → **Memoization** → **Tabulation** → **Space Optimization!**

1 **STAGE 1: FORMULATION (S-C-R-B)**

- **State ('S'):** `dp[i]` = max profit at index `i`.
- **Choice ('C'):** Pick element vs Skip element.
- **Recurrence ('R'):** `dp[i] = max(dp[i-1], val[i] + dp[i-2])`.
- **Base Case ('B'):** `dp[0] = val[0], dp[1] = max(val[0], val[1])`.

2 **STAGE 2: OPTIMIZATION (R-M-T-S)**

- **Recursion ('R'):** Brute force tree search.
- **Memoization ('M'):** Cache answers in `memo[]`.
- **Tabulation ('T'):** Iterative array loop `for(i=2..n)`.
- **Space Opt ('S'):** Compress array to `prev1` & `prev2`!

🔪 **SC-RBR-MTS FRAMEWORK REUSABLE JAVA TRANSFORMATION**

```
// 1. MEMOIZATION (Top-Down)
int solveMemo(int n, Integer[] memo) {
    if (n ≤ 1) return n;
    if (memo[n] ≠ null) return memo[n];
    return memo[n] = solveMemo(n - 1, memo) + solveMemo(n - 2, memo);
}

// 2. TABULATION (Bottom-Up)
int solveTab(int n) {
    if (n ≤ 1) return n;
    int[] dp = new int[n + 1];
    dp[0] = 0; dp[1] = 1;
    for (int i = 2; i ≤ n; i++) dp[i] = dp[i - 1] + dp[i - 2];
    return dp[n];
}

// 3. SPACE OPTIMIZATION (O(1) Space)
int solveSpaceOpt(int n) {
    if (n ≤ 1) return n;
    int prev2 = 0, prev1 = 1;
    for (int i = 2; i ≤ n; i++) {
        int curr = prev1 + prev2;
        prev2 = prev1; prev1 = curr;
    }
    return prev1;
}
```

🔪 **SPACE OPTIMIZATION RULE:** If state `dp[i]` depends **ONLY** on previous **\$K\$** states (`dp[i-1]`, `dp[i-2]`), replace array with **\$K\$** primitive variables!



💡 AHA MOMENT #3: THE CHOICE PARADIGM (SKIP VS TAKE)

In 1D DP problems like **House Robber**, at each step `i` you have a binary choice:

- **Option A (Skip current):** Profit remains `dp[i-1]`.
- **Option B (Take current):** Profit becomes `val[i] + dp[i-2]` (cannot rob adjacent house!).

Recurrence: `dp[i] = Math.max(dp[i-1], val[i] + dp[i-2])`

📌 1D DP RECURRENCE RELATION FORMULAS

CLIMBING STAIRS (LC 70): `dp[i] = dp[i-1] + dp[i-2]` (Reach step i from step i-1 or step i-2) **HOUSE ROBBER (LC 198):** `dp[i] = Math.max(dp[i-1], nums[i] + dp[i-2])` (Skip house i vs Rob house i) **DECODE WAYS (LC 91):** `dp[i] = (valid 1-digit ? dp[i-1] : 0) + (valid 2-digit ? dp[i-2] : 0)`

⚠️ **COMMON BUG:** Forgetting to initialize base cases (`dp[0]`, `dp[1]`) correctly leads to `ArrayIndexOutOfBoundsException` on small input arrays!

💡 **AHA MOMENT #4: GRID PATH ACCUMULATION & ROW SPACE COMPRESSION**

In a 2D Grid, moving only ****Right**** and ****Down**** means cell `(r, c)` can **ONLY** be reached from Top `(r-1, c)` or Left `(r, c-1)`. Since row `r` depends **ONLY** on row `r-1`, compress $O(R \cdot C)$ 2D table into a single 1D row array `dp[c]`!

📐 **2D GRID DP RECURRENCE FORMULAS & SPACE COMPRESSION**

UNIQUE PATHS (LC 62): `dp[r][c] = dp[r-1][c] + dp[r][c-1]` (Top paths + Left paths) Space-Optimized: `dp[c] = dp[c] + dp[c-1]` **MINIMUM PATH SUM (LC 64):** `dp[r][c] = grid[r][c] + Math.min(dp[r-1][c], dp[r][c-1])` Space-Optimized: `dp[c] = grid[r][c] + Math.min(dp[c], dp[c-1])`

🚀 **KEY TAKEAWAY:** Initialize top row and leftmost column first as base cases before running inner loops!

💡 AHA MOMENT #5: INNER LOOP DIRECTION (BACKWARDS VS FORWARDS)

- **0/1 Knapsack (Use item AT MOST ONCE):** Inner loop runs **BACKWARDS** (W to w_i) so `dp[w - weight]` references values from the PREVIOUS item, preventing double-counting!
- **Unbounded Knapsack (Use item INFINITE TIMES):** Inner loop runs **FORWARDS** (w_i to W) allowing multiple inclusions of the same item!

📊 0/1 KNAPSACK VS UNBOUNDED KNAPSACK MATRIX

Knapsack Variant	Item Re-Use Allowed?	Inner Loop Direction	Target Problems
0/1 Knapsack	❌ No (Pick once or zero)	⬅️ Backwards ('W' down to 'w_i')	Partition Subset Sum (LC 416), Target Sum (LC 494)
Unbounded Knapsack	✅ Yes (Pick infinite times)	➡️ Forwards ('w_i' up to 'W')	Coin Change (LC 322), Coin Change II (LC 518)

⚠️ **CRITICAL BUG:** Running the inner loop forwards in a 0/1 Knapsack problem accidentally turns it into an Unbounded Knapsack!

💡 **AHA MOMENT #6: 2D STRING MATRIX COMPARISON**

For two strings `text1` (len \$M\$) and `text2` (len \$N\$), construct a 2D matrix `dp[i][j]` representing substring matches up to `text1[i-1]` and `text2[j-1]`.

1 LONGEST COMMON SUBSEQUENCE (LC 1143)

```

if (text1.charAt(i - 1) == text2.charAt(j - 1)) {
    dp[i][j] = 1 + dp[i - 1][j - 1]; // Character match!
} else {
    dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]); // Skip char
}

```

2 EDIT DISTANCE (LC 72)

```

if (word1.charAt(i - 1) == word2.charAt(j - 1)) {
    dp[i][j] = dp[i - 1][j - 1]; // Zero edit cost
} else {
    // 1 + min(Insert, Delete, Replace)
    dp[i][j] = 1 + Math.min(dp[i][j - 1],
                            Math.min(dp[i - 1][j], dp[i - 1][j - 1]));
}

```

🚩 **KEY TAKEAWAY:** Base cases `dp[i][0] = i` (deletions) and `dp[0][j] = j` (insertions) initialize string matching grids!

💡 **AHA MOMENT #7: LAST BALLOON CHOICE & TREE POSTORDER DP**

- **Interval DP (Burst Balloons):** Instead of picking which balloon to burst first, pick which balloon k bursts ****LAST**** in interval `[left, right]` so boundaries `arr[left-1]` and `arr[right+1]` remain fixed!
- **Tree DP (House Robber III):** Postorder DFS returning `int[] {robRoot, skipRoot}` up the binary tree!

1 BURST BALLOONS INTERVAL DP (LC 312)

```
for (int k = left; k ≤ right; k++) {
    int coins = arr[left - 1] * arr[k] * arr[right + 1];
    dp[left][right] = Math.max(dp[left][right],
        coins + dp[left][k - 1] + dp[k + 1][right]);
}
```

2 HOUSE ROBBER III TREE DP (LC 337)

```
private int[] dfs(TreeNode root) {
    if (root == null) return new int[]{0, 0};
    int[] left = dfs(root.left), right = dfs(root.right);
    int rob = root.val + left[1] + right[1];
    int skip = Math.max(left[0], left[1]) + Math.max(right[0],
        right[1]);
    return new int[]{rob, skip};
}
```

🔑 **KEY TAKEAWAY:** Tree DP uses bottom-up postorder traversal to pass optimal state pairs `[rob, skip]` from leaves up to root!

💡 **AHA MOMENT #8: PATIENCE SORTING & BINARY SEARCH SPEEDUP**

Standard LIS DP runs in $O(N^2)$ time (' $dp[i] = 1 + \max(dp[j])$ ' for $j < i$).

By maintaining an array 'tails[]' storing the smallest tail of all increasing subsequences of length $i+1$, use **Binary Search** ('`Arrays.binarySearch`') to update 'tails[]' in **$O(N \log N)$ time!**

🔪 **LIS $O(N \log N)$ BINARY SEARCH IMPLEMENTATION**

```
public int lengthOfLIS(int[] nums) {
    int[] tails = new int[nums.length];
    int size = 0;
    for (int num : nums) {
        int i = 0, j = size;
        while (i < j) {
            int mid = (i + j) / 2;
            if (tails[mid] < num) i = mid + 1;
            else j = mid;
        }
        tails[i] = num;
        if (i == size) size++; // Expanded longest subsequence!
    }
    return size;
}
```

🔪 **KEY TAKEAWAY:** 'tails[]' is always strictly sorted, allowing binary search to replace smaller tail elements greedily!

DP INTERVIEW DECISION TREE (MERMAID)

PRINTABLE DYNAMIC PROGRAMMING CHEAT SHEET

Pattern Type	Target Problem	Recurrence Formula	Time & Space
1D DP	House Robber (LC 198)	<code>dp[i] = max(dp[i-1], val + dp[i-2])</code>	Time: $O(N)$, Space: $O(1)$
2D Grid DP	Unique Paths (LC 62)	<code>dp[r][c] = dp[r-1][c] + dp[r][c-1]</code>	Time: $O(R \cdot C)$, Space: $O(C)$
0/1 Knapsack	Partition Subset Sum (LC 416)	<code>dp[w] = dp[w] dp[w - num]</code> (Backwards)	Time: $O(N \cdot W)$, Space: $O(W)$
Unbounded Knapsack	Coin Change (LC 322)	<code>dp[a] = min(dp[a], 1 + dp[a - coin])</code> (Forwards)	Time: $O(N \cdot A)$, Space: $O(A)$
LCS String DP	Longest Common Subseq (LC 1143)	<code>if match: 1+dp[i-1][j-1]; else: max(top, left)</code>	Time: $O(M \cdot N)$, Space: $O(N)$
LIS Binary Search	Longest Increasing Subseq (LC 300)	Patience Sort Binary Search in <code>tails[]</code>	Time: $O(N \log N)$, Space: $O(N)$


FAANG COACH TIP: Print this page and review it 10 minutes before your technical interview!

1. 1D DP SPACE-OPTIMIZED TEMPLATE

```
int dp1D(int[] nums) {
    int prev2 = 0, prev1 = 0;
    for (int num : nums) {
        int curr = Math.max(prev1, num + prev2);
        prev2 = prev1; prev1 = curr;
    }
    return prev1;
}
```

2. 2D GRID DP SPACE-OPTIMIZED TEMPLATE

```
int dpGrid(int[][] grid) {
    int rows = grid.length, cols = grid[0].length;
    int[] dp = new int[cols];
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (c > 0) dp[c] = grid[r][c] + Math.min(dp[c], dp[c - 1]);
            else dp[0] += grid[r][0];
        }
    }
    return dp[cols - 1];
}
```

3. 0/1 KNAPSACK TEMPLATE (BACKWARDS)

```
boolean knapsack01(int[] nums, int target) {
    boolean[] dp = new boolean[target + 1];
    dp[0] = true;
    for (int num : nums) {
        for (int w = target; w ≥ num; w--) { // Backwards!
            dp[w] = dp[w] || dp[w - num];
        }
    }
    return dp[target];
}
```

4. UNBOUNDED KNAPSACK TEMPLATE (FORWARDS)

```
int knapsackUnbounded(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1); dp[0] = 0;
    for (int a = 1; a ≤ amount; a++) {
        for (int c : coins) {
            if (a - c ≥ 0) dp[a] = Math.min(dp[a], 1 + dp[a - c]); //
        }
    }
    return dp[amount] > amount ? -1 : dp[amount];
}
```

🚀 **REUSABLE CODE:** Copy-paste these verified templates directly into your interview whiteboard or IDE!



5. LCS STRING MATCHING TEMPLATE

```
int lcs(String s1, String s2) {
    int m = s1.length(), n = s2.length();
    int[][] dp = new int[m + 1][n + 1];
    for (int i = 1; i ≤ m; i++) {
        for (int j = 1; j ≤ n; j++) {
            if (s1.charAt(i-1) == s2.charAt(j-1)) dp[i][j] = 1 + dp[i-1][j-1];
            else dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);
        }
    }
    return dp[m][n];
}
```

6. LIS $O(N \log N)$ BINARY SEARCH TEMPLATE

```
int lis(int[] nums) {
    int[] tails = new int[nums.length];
    int size = 0;
    for (int x : nums) {
        int i = 0, j = size;
        while (i < j) {
            int m = (i + j) / 2;
            if (tails[m] < x) i = m + 1; else j = m;
        }
        tails[i] = x; if (i == size) size++;
    }
    return size;
}
```

🚀 **KEY TAKEAWAY:** Memorize these core templates to solve 95% of FAANG Dynamic Programming problems!



1. LEETCODE 70 — CLIMBING STAIRS

Easy ★★★★★ 1D DP / Fibonacci State [Amazon](#) [Google](#)**Problem:**

Distinct ways to climb N steps taking 1 or 2 steps at a time.

```
public int climbStairs(int n) {
    if (n ≤ 2) return n;
    int prev2 = 1, prev1 = 2;
    for (int i = 3; i ≤ n; i++) {
        int curr = prev1 + prev2;
        prev2 = prev1; prev1 = curr;
    }
    return prev1;
}
```

2. LEETCODE 746 — MIN COST CLIMBING STAIRS

Easy ★★★★★ 1D DP Min Cost [Amazon](#)**Problem:**

Minimum cost to reach top of staircase paying `cost[i]` per step.

```
public int minCostClimbingStairs(int[] cost) {
    int prev2 = cost[0], prev1 = cost[1];
    for (int i = 2; i < cost.length; i++) {
        int curr = cost[i] + Math.min(prev1, prev2);
        prev2 = prev1; prev1 = curr;
    }
    return Math.min(prev1, prev2);
}
```

💡 **AHA MOMENT (LC 70):** Space-optimizing to `prev1` & `prev2` reduces space from $O(N)$ to $O(1)$!

3. LEETCODE 198 — HOUSE ROBBER **Medium ★★★★★** 1D DP Non-Adjacent Choice [Amazon](#) [Meta](#)**Problem:**

Maximum money robbed without robbing adjacent houses.

```
public int rob(int[] nums) {
    int prev2 = 0, prev1 = 0;
    for (int num : nums) {
        int curr = Math.max(prev1, num + prev2);
        prev2 = prev1; prev1 = curr;
    }
    return prev1;
}
```

4. LEETCODE 91 — DECODE WAYS **Medium ★★★★★** String Decoding 1D DP [Amazon](#) [Meta](#)**Problem:**

Total number of ways to decode string of digits into letters ('A'-'Z').

```
public int numDecodings(String s) {
    if (s == null || s.length() == 0 || s.charAt(0) == '0') return 0;
    int n = s.length(), prev2 = 1, prev1 = 1;
    for (int i = 1; i < n; i++) {
        int curr = 0;
        int oneDigit = s.charAt(i) - '0';
        int twoDigits = Integer.parseInt(s.substring(i - 1, i + 1));
        if (oneDigit != 0) curr += prev1;
        if (twoDigits ≥ 10 && twoDigits ≤ 26) curr += prev2;
        prev2 = prev1; prev1 = curr;
    }
    return prev1;
}
```

 **AHA MOMENT (LC 91):** Leading zeroes ``0`` cannot form valid decodings and immediately return 0!



5. LEETCODE 322 — COIN CHANGE



Medium ★★★★★ Unbounded Knapsack Min Coins

Amazon

Meta

Problem:

Fewest number of coins needed to make up target 'amount'.

```
public int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1); dp[0] = 0;
    for (int a = 1; a ≤ amount; a++) {
        for (int c : coins) {
            if (a - c ≥ 0) dp[a] = Math.min(dp[a], 1 + dp[a - c]);
        }
    }
    return dp[amount] > amount ? -1 : dp[amount];
}
```

6. LEETCODE 518 — COIN CHANGE II



Medium ★★★★★ Unbounded Knapsack Combinations

Amazon

Problem:

Number of combinations to make up target 'amount'.

```
public int change(int amount, int[] coins) {
    int[] dp = new int[amount + 1];
    dp[0] = 1;
    for (int c : coins) {
        for (int a = c; a ≤ amount; a++) {
            dp[a] += dp[a - c];
        }
    }
    return dp[amount];
}
```

**AHA MOMENT (LC 518):** Placing 'coins' in the OUTER loop counts COMBINATIONS (order doesn't matter).

7. LEETCODE 62 — UNIQUE PATHS

Medium
★★★★★
2D Grid DP Space-Optimized
Amazon
Meta

Problem:
 Number of possible unique paths from top-left to bottom-right cell in $M \times N$ grid.

```

public int uniquePaths(int m, int n) {
    int[] dp = new int[n];
    Arrays.fill(dp, 1);
    for (int r = 1; r < m; r++) {
        for (int c = 1; c < n; c++) {
            dp[c] += dp[c - 1];
        }
    }
    return dp[n - 1];
}

```

8. LEETCODE 64 — MINIMUM PATH SUM

Medium
★★★★★
2D Grid Min Cost
Amazon

Problem:
 Path from top-left to bottom-right minimizing sum of numbers along path.

```

public int minPathSum(int[][] grid) {
    int rows = grid.length, cols = grid[0].length;
    int[] dp = new int[cols];
    dp[0] = grid[0][0];
    for (int c = 1; c < cols; c++) dp[c] = dp[c - 1] + grid[0][c];
    for (int r = 1; r < rows; r++) {
        dp[0] += grid[r][0];
        for (int c = 1; c < cols; c++) {
            dp[c] = grid[r][c] + Math.min(dp[c], dp[c - 1]);
        }
    }
    return dp[cols - 1];
}

```

★ **KEY TAKEAWAY:** Space-optimizing 2D Grid DP into a single 1D row array cuts space from $O(M \times N)$ to $O(N)$!



9. LEETCODE 416 — PARTITION EQUAL SUBSET SUM

Medium ★★★★★ 0/1 Knapsack Subset Sum Amazon Meta

Problem:

Determine if array can be partitioned into two subsets with equal sum.

```
public boolean canPartition(int[] nums) {
    int sum = 0;
    for (int num : nums) sum += num;
    if (sum % 2 != 0) return false;
    int target = sum / 2;
    boolean[] dp = new boolean[target + 1];
    dp[0] = true;
    for (int num : nums) {
        for (int w = target; w >= num; w--) { // Backwards loop for 0/1!
            dp[w] = dp[w] || dp[w - num];
        }
    }
    return dp[target];
}
```

10. LEETCODE 1143 — LONGEST COMMON SUBSEQUENCE

Medium ★★★★★ 2D String Subsequence Matrix Amazon Google

Problem:

Length of longest common subsequence between `text1` and `text2`.

```
public int longestCommonSubsequence(String text1, String text2) {
    int m = text1.length(), n = text2.length();
    int[][] dp = new int[m + 1][n + 1];
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (text1.charAt(i - 1) == text2.charAt(j - 1)) {
                dp[i][j] = 1 + dp[i - 1][j - 1];
            } else {
                dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
    }
    return dp[m][n];
}
```

**AHA MOMENT (LC 416):** Target sum is `totalSum / 2`. If `totalSum` is odd, equal partition is mathematically impossible!

11. LEETCODE 300 — LONGEST INCREASING SUBSEQUENCE

 **Medium ★★★★★** **Patience Sort** $O(N \log N)$  

Problem:

Length of longest strictly increasing subsequence in integer array.

```
public int lengthOfLIS(int[] nums) {
    int[] tails = new int[nums.length];
    int size = 0;
    for (int num : nums) {
        int i = 0, j = size;
        while (i < j) {
            int mid = (i + j) / 2;
            if (tails[mid] < num) i = mid + 1;
            else j = mid;
        }
        tails[i] = num;
        if (i == size) size++;
    }
    return size;
}
```

12. LEETCODE 72 — EDIT DISTANCE

 **Hard ★★★★★** **2D String Transformation DP**  

Problem:

Minimum ops (Insert, Delete, Replace) to convert 'word1' to 'word2'.

```
public int minDistance(String word1, String word2) {
    int m = word1.length(), n = word2.length();
    int[][] dp = new int[m + 1][n + 1];
    for (int i = 0; i ≤ m; i++) dp[i][0] = i;
    for (int j = 0; j ≤ n; j++) dp[0][j] = j;
    for (int i = 1; i ≤ m; i++) {
        for (int j = 1; j ≤ n; j++) {
            if (word1.charAt(i - 1) == word2.charAt(j - 1)) {
                dp[i][j] = dp[i - 1][j - 1];
            } else {
                dp[i][j] = 1 + Math.min(dp[i][j - 1],
                    Math.min(dp[i - 1][j], dp[i - 1][j - 1]));
            }
        }
    }
    return dp[m][n];
}
```

 **FAANG COACH TIP:** Master LCS and Edit Distance formulas to solve 90% of string DP interview problems!

★ THE 4 GOLDEN LAWS OF DYNAMIC PROGRAMMING

- Define State First:** What does `dp[i]` or `dp[r][c]` represent in plain English?
- O/1 Knapsack Backwards Loop:** Inner loop runs `$W \to w_i$` backwards to prevent item re-use.
- Unbounded Knapsack Forwards Loop:** Inner loop runs `$w_i \to W$` forwards allowing item re-use.
- LIS $O(N \log N)$ Binary Search:** Maintain `tails[]` array for patience sorting!

🚫 WHEN DYNAMIC PROGRAMMING FAILS (DO NOT USE!)

- No Overlapping Subproblems:** If all subproblems are unique → Use ****Divide & Conquer / MergeSort****!
- Local Choice Always Optimal:** If picking local max always gives global max → Use ****Greedy****!

📁 NEETCODE & BLIND 75 CONFIDENCE TRACKER

Easy LC 70 Climbing Stairs	Easy LC 746 Min Cost Stairs	Med LC 198 House Robber
Med LC 213 House Robber II	Med LC 91 Decode Ways	Med LC 322 Coin Change
Med LC 518 Coin Change II	Med LC 62 Unique Paths	Med LC 64 Min Path Sum
Med LC 416 Subset Sum	Med LC 1143 LCS	Med LC 300 LIS
Hard LC 72 Edit Distance	Hard LC 312 Burst Balloons	

👉 **Next Master Topic: Topic 15 → Greedy Algorithms Masterclass**

🇬🇧 **TOPIC 14 DYNAMIC PROGRAMMING MASTERCLASS COMPLETE!** Turn the page to **Topic 15: Greedy Algorithms Masterclass!**